**Đại học Quốc gia Thành phố Hồ Chí Minh**

**Đại học Khoa học tự nhiên**

*Khoa Công nghệ thông tin*


# HOMEWORK

## API TESTING

Subject **SOFTWARE TESTING**

Class 22KTPM3

Student **22127374 – Lê Thanh Tâm**

*Ho Chi Minh City, 2025*

# Table of Contents

# 1 Group Task Allocation

| Fullname | StudentID | API Task Allocation |
|---|---|---|
| Lương Hoàng Dung | 22127076 | `POST/products`<br>`PUT/products/productId`<br>`DELETE/products/productId` |
| Lê Thanh Tâm | 22127374 | `GET /products/search`<br>`POST /favorites`<br>`GET & DELETE /favorites/{favoriteId}` |
| Nguyễn Lâm Nhã Uyên | 22127445 | `POST/users/login`<br>`PUT/users/userID`<br>`GET/users/me` |
| Lê Nguyễn Yến Vy | 22127466 | `POST /payment/check`<br>`POST /invoices`<br>`PUT /invoices` |

# 2 Introduction

In the landscape of modern software development, Application Programming Interfaces (APIs) serve as the critical backbone for countless applications, facilitating data exchange and business logic between different system components. The quality and reliability of these APIs are paramount to the overall performance, security, and user experience of the software. Consequently, rigorous API testing has become an indispensable phase in the quality assurance lifecycle.

The primary goal of this exercise is to apply systematic testing methodologies to identify defects, validate functionality, and ensure the robustness of the application's core backend services. The scope of work involved designing and executing a comprehensive suite of over 90 test cases across three API endpoints, reporting all identified bugs, and integrating a selection of these tests into an automated Continuous Integration (CI) workflow.

To provide a clear and structured overview of the project, this document is organized into several key sections. It begins with an Overview detailing the project's objectives, scope, and the tools employed. The report then delves into the API Test Design and Execution, which elaborates on the testing strategies and outcomes for each specific API. Subsequently, the Bug Reporting section documents the defects discovered. The implementation of test automation is covered in the CI/CD Integration section. The report also discusses the use of AI/LLM Tools to enhance the testing process and concludes with a summary of the key findings and lessons learned.

# 3 Overview

## 3.1 Project Objective

The fundamental objective of this assignment is to conduct a thorough and systematic testing process on the Application Programming Interface (API) of a real-world software project. This exercise is

designed to bridge the gap between theoretical testing concepts and practical application, focusing on ensuring the functionality, reliability, and robustness of the application's backend services.

The key goals encompassed within this project are:

- **Analysis and Test Design:** To dissect the provided API specifications and apply formal black-box testing techniques, such as Domain Testing (including Equivalence Class Partitioning and Boundary Value Analysis) and State Transition Testing, to design a comprehensive suite of test cases.
- **Test Execution and Defect Management:** To execute the designed test cases using a professional API testing platform, meticulously identify deviations from expected outcomes, and report all discovered defects in a structured manner using a formal bug tracking system.
- **Automation and CI/CD Integration:** To automate a critical subset of the API tests and integrate them into a Continuous Integration/Continuous Deployment (CI/CD) pipeline. This aims to establish a safety net that automatically validates core functionalities upon new code changes, thereby preventing regression bugs.
- **Reporting and Documentation:** To produce professional documentation detailing the entire testing process, including the test plan, test cases, bug reports, and a summary of the outcomes.

Ultimately, this assignment serves to demonstrate a comprehensive understanding of the API testing lifecycle, from initial planning to automated execution and final reporting.

## 3.2 Software Under Test (SUT)

The application selected for this testing endeavor is "The Toolshop" an open-source project specifically developed to serve as a practice ground for software quality assurance activities.

- **Project Name:** The Toolshop
- **Technology Stack:** The system is built upon a modern architecture, featuring a RESTful API backend that serves data to an Angular-based single-page application (SPA) front-end.
- **Source Code:** https://github.com/testsmith-io/practice-software-testing/
- **Tested Version:** sprint5-with-bugs

The sprint5-with-bugs version was deliberately chosen for this project. Testing a version known to contain pre-existing bugs provides a realistic industry scenario. It allows for the practical application of the full defect lifecycle: not only discovering new, unknown issues but also confirming and documenting known ones. This process is invaluable for honing analytical and reporting skills.

## 3.3 Scope of Testing

To ensure a focused and impactful testing effort, the scope was narrowed to three critical groups of API endpoints. These were selected based on their high business value and importance to the core user experience, such as searching for products and managing user-specific data like favorites.
The specific APIs included in the scope are detailed in Table 3.3.

| No. | API Name | Endpoint & Methods | Description of Functionality |
|---|---|---|---|
| 1 | API_1 | `GET /products/search` | Enables users to search the product catalog based on keywords and filter criteria. This is a primary function for product discovery. |
| 2 | API_2 | `POST /favorites` | Allows authenticated users to add a specific product to their personal list of favorites, a key feature for user engagement. |
| 3 | API_3 | `GET /favorites/{favoriteId}` `DELETE /favorites/{favoriteId}` | Provides mechanisms for authenticated users to view the details of a specific favorite item and to remove an item from their list. |

This scope is designed to cover essential user journeys and interactions with the system's core data and business logic.

## 3.4 Tools Used

A curated set of industry-standard tools was employed to facilitate a professional and efficient testing workflow. Each tool was selected for its specific role in the testing lifecycle, as described in Table 3.4.

| Tool | Purpose and Role in the Project |
|---|---|
| **Postman** | Utilized as the primary integrated environment for all API testing activities. Its interface was used for the initial exploration of APIs, the manual execution of tests, and the crucial task of writing JavaScript-based assertions (`pm.test`) to programmatically validate responses. |
| **Newman** | Served as the command-line interface (CLI) runner for Postman collections. Newman was essential for enabling automation, allowing the entire test suite to be executed headlessly from the command line, which is a prerequisite for CI/CD integration. |
| **GitHub & GitHub Actions** | GitHub was used for version control of all project artifacts, including test documentation. GitHub Actions provided the cloud-based orchestration engine for the CI/CD pipeline, automatically triggering the Newman test scripts on every code push to the main branch. |
| **Microsoft Excel** | Used for the detailed documentation and management of the comprehensive test case suite. It provided a structured format for outlining test steps, expected results, and tracking the status of each case. |

# 4 API Test Design and Execution

This section provides a detailed breakdown of the testing process for the three selected API functional groups. The overall strategy involved a multi-faceted approach: beginning with a thorough analysis of each endpoint's specification, followed by the application of formal testing techniques to design a comprehensive test suite. These test cases were then executed manually to identify defects, and a curated subset was selected for inclusion in an automated CI/CD pipeline to ensure continuous quality control.

## 4.1 API 1: `GET /products/search` - Product Search

- **Endpoint:** `GET /products/search`
- **Description:** Searches for products based on a query string.
- **Authentication:** Not required.
- **Query Parameters:**
  - `q` (string): The search keyword.
- **Responses:**
  - `200 OK`: Successful search. The body contains an array of product objects, which can be empty if no results are found.
  - `400 Bad Request`: Expected for invalid or malformed requests.

### 4.1.1 Testing Techniques Applied

**Domain Testing** was the primary technique used for this stateless, data-centric API, focusing on the `q` parameter.

- **Equivalence Class Partitioning (ECP):**
  - *Valid Classes:* Keywords expected to return results (e.g., "wrench"), keywords returning no results (e.g., "nonexistentproduct"), an empty string.
  - *Invalid Classes:* Input with special characters to test for sanitization (e.g., `<script>`), path traversal characters (`../`), and SQL injection payloads (`' OR 1=1`). These were tested in cases like **SEARCH_TC18**.
- **Boundary Value Analysis (BVA):**
  - An empty string was tested as a boundary condition.
  - A very long string was used to test for performance degradation or potential buffer overflow issues.

### 4.1.2 Test Case Summary and CI/CD Selection

For the CI/CD pipeline, a lean set of test cases was chosen to validate the search API's core behavior quickly.

| Test Case Type | Description | Rationale for CI/CD Inclusion |
|---|---|---|
| Happy Path | Search with a valid keyword expecting results. | Confirms the primary functionality is working. |
| Common Case | Search expecting zero results. | Ensures the API handles empty result sets gracefully. |
| Bug Regression | Search with invalid path traversal characters (**SEARCH_TC18**). | Verifies a previously found security flaw (**BUG-002**) does not reappear. |

### 4.1.3 Execution Results and Illustration

Execution in Postman revealed several issues, including security vulnerabilities where the API failed to reject malicious inputs.

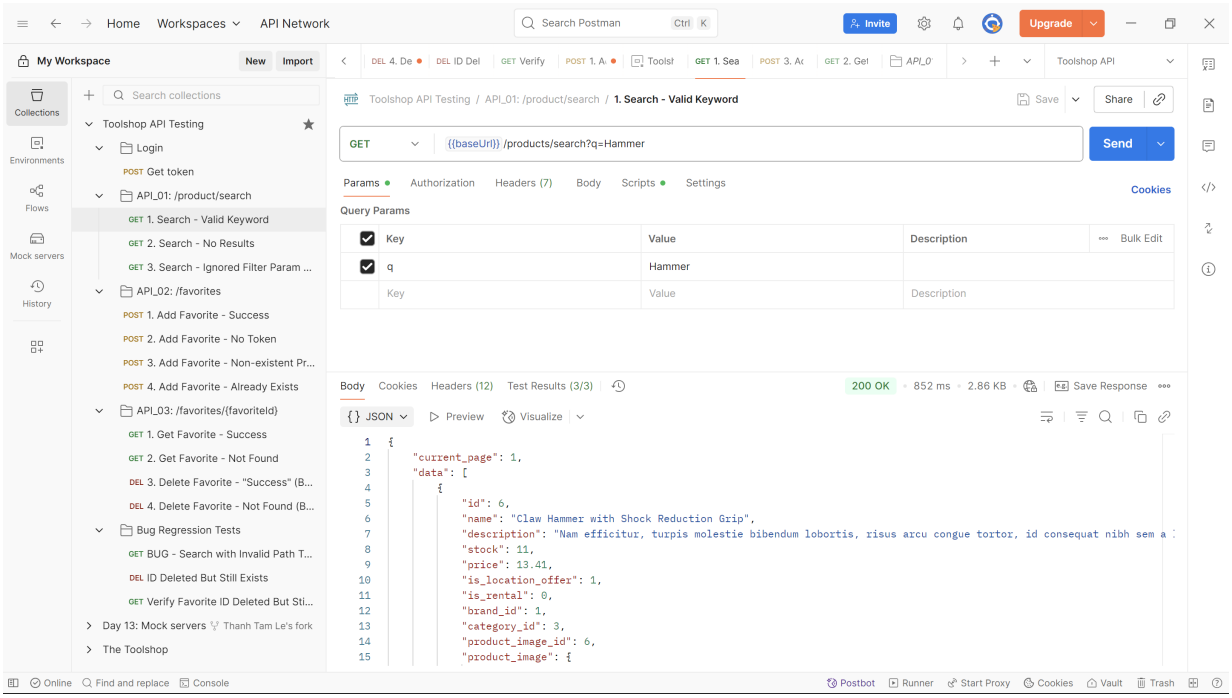



Figure 1: Successful execution of a valid search query in Postman.

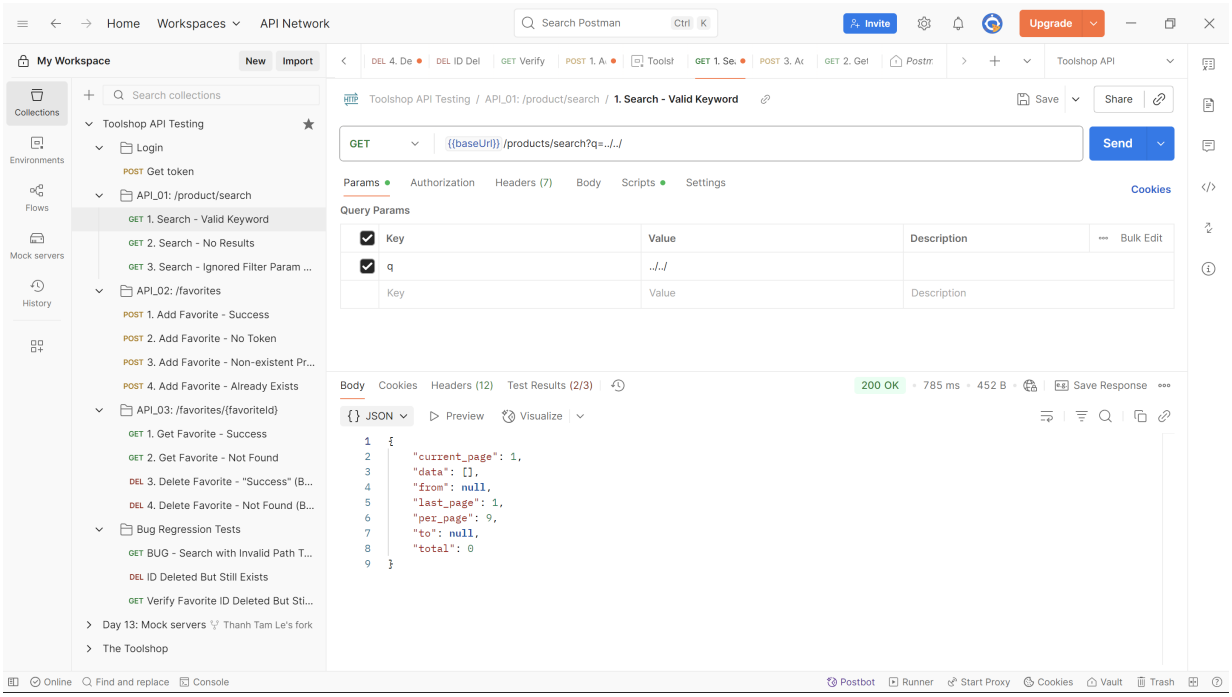



Figure 2: Execution of SEARCH_TC18, which identified a path traversal vulnerability.

## 4.2 API 2: `POST /favorites` - Add Product to Favorites

This API is a critical component of user engagement, allowing authenticated users to save products for future reference.

### 4.2.1 API Specification

- **Endpoint:** `POST /favorites`
- **Description:** Creates a new "favorite" entry linking a user to a product.
- **Authentication:** Required (Bearer Token).
- **Request Body:** JSON object with a required `product_id` (integer).
- **Responses:** `201 Created` (Success), `401 Unauthorized` (Auth Failure), `422 Unprocessable Content` (Validation Error).

### 4.2.2 Testing Techniques Applied

1. **Domain Testing:** This was crucial for validating the `product_id` field.
   - **ECP (Data Type Validation):** A wide range of invalid data types were tested, including string (**FAV_TC11**), text (**FAV_TC12**), float (**FAV_TC13**), null (**FAV_TC14**), boolean (**FAV_TC15**), and array (**FAV_TC16**).
   - **ECP (Value Validation):** Non-existent (**FAV_TC08**) and negative (**FAV_TC10**) IDs were tested.

- **BVA:** Boundary values such as '0' (**FAV_TC09**) and a very large number (**FAV_TC17**) were tested.

**2. State Transition Testing:** This technique was essential for validating the business logic of the feature.

- **States:** 1. Not Favorited, 2. Favorited.
- **Transitions:**
  - *Add New (Not Favorited → Favorited):* The primary success scenario, tested in **FAV_TC01**.
  - *Add Duplicate (Favorited → Favorited):* Validated that adding an existing favorite results in an error, as tested in **FAV_TC06**.
  - *Re-Add (Favorited → Not Favorited → Favorited):* This test (**FAV_TC07**) was designed but was **BLOCKED** by a critical bug in the DELETE API.

#### 4.2.3 Test Case Summary and CI/CD Selection

Out of 31 test cases designed, a strategic subset was chosen for the CI pipeline to ensure rapid validation of core logic and security.

| Test Case ID | Description | Rationale for CI/CD Inclusion |
|---|---|---|
| **FAV_TC01** | Add a valid, existing product. | **Happy Path**: Ensures the fundamental purpose of the API works correctly. |
| **FAV_TC03** | Attempt to add a favorite without an auth token. | **Security Check**: Verifies that authentication is enforced. |
| **FAV_TC06** | Attempt to add a product that is already a favorite. | **Business Logic**: Ensures a key business rule (preventing duplicates) is handled. |
| **FAV_TC08** | Attempt to add a favorite with a non-existent product ID. | **Bug Regression (BUG-004)**: Checks for regressions of a known data validation issue. |

#### 4.2.4 Execution Results and Illustration

Execution revealed defects such as incorrect status codes and unexpected behavior with invalid data types.

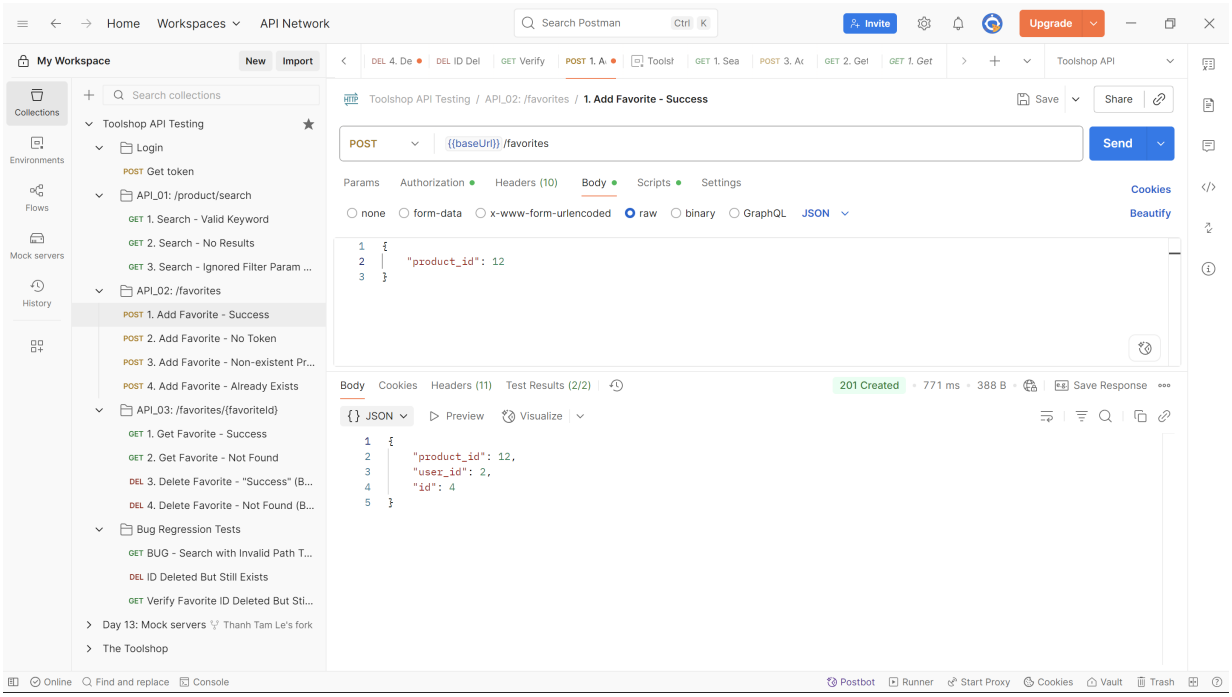



Figure 3: Successful execution of a positive test case (FAV_TC01).

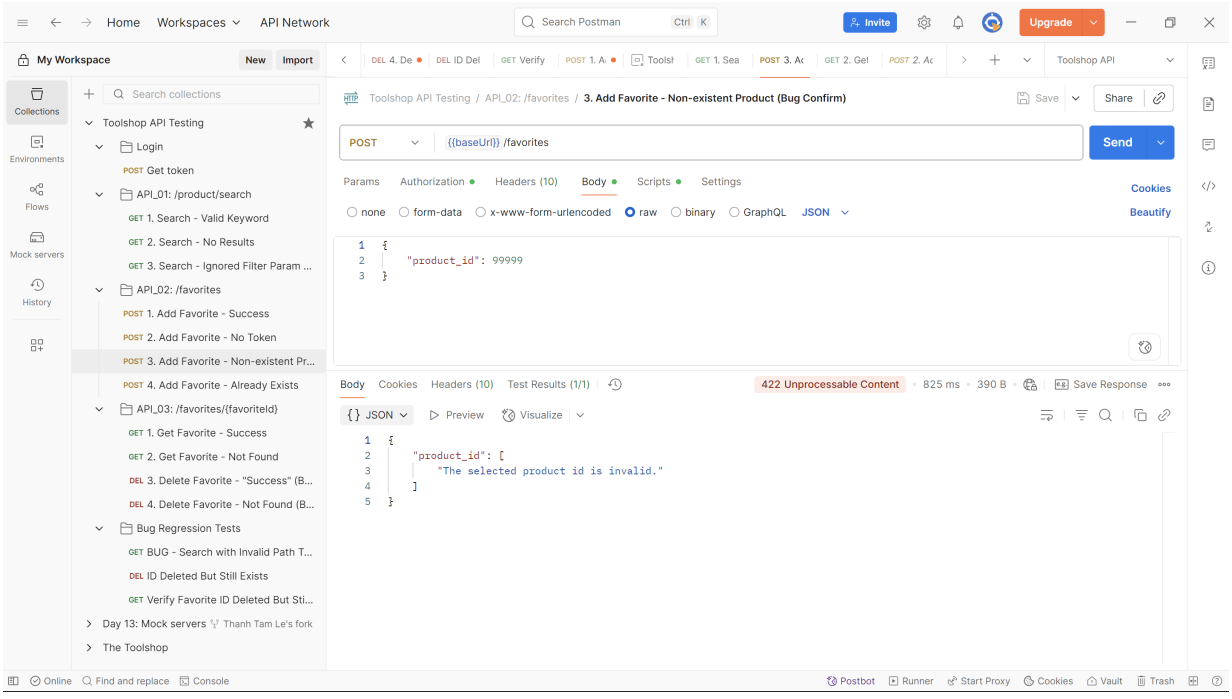



Figure 4: Identified a bug, returning a 422 instead of the expected 404 status code.

## 4.3 API 3: `GET & DELETE /favorites/{favoriteId}` - Get and Delete a Favorite Item

These endpoints manage the lifecycle of a favorite item after its creation. Their correct functioning is critical for user data management.

### 4.3.1 API Specification

- **Endpoint:** `/favorites/{favoriteId}`
- **Description:** Retrieves or deletes a single favorite item by its unique ID.
- **Authentication:** Required (Bearer Token).
- **Methods:**
  - `GET`: Retrieves item details. Responds with `200 OK` (Success) or `404 Not Found`.
  - `DELETE`: Deletes the item. Responds with `204 No Content` (Success) or `404 Not Found`.

### 4.3.2 Testing Techniques Applied

**1. Domain Testing:** This was applied to the `{favoriteId}` path parameter.

- **ECP/BVA:** Tests included valid IDs (**FAV_GET_TC01**), non-existent IDs (**FAV_GET_TC02**), '0' (**FAV_GET_TC03**), negative values (**FAV_GET_TC04**), and non-integer values (**FAV_GET_TC05**). These tests uncovered bugs where the API returned incorrect status codes or an internal server error (**BUG-009** to **BUG-012**).
- **Authorization:** A critical test case, **FAV_DEL_TC19**, was designed to verify that one user cannot delete another user's favorite item, which uncovered a critical authorization bypass flaw (**BUG-013**).

**2. State Transition Testing:** This was the most critical technique for these endpoints, as they directly manipulate the state of a favorite item.

- **States:** 1. Exists, 2. Deleted.
- **Transitions:**
  - *Retrieve Existing Item (Exists → Exists):* Validated with **FAV_GET_TC01**.
  - *Delete Existing Item (Exists → Deleted):* The primary DELETE scenario, tested in **FAV_DEL_TC12**. While the API returned a `204 No Content` status, further testing proved the state transition did not actually occur.
  - *Retrieve Deleted Item (Deleted → Deleted):* This critical test, **FAV_GET_TC10**, was designed to verify that a GET request after a DELETE returns a `404 Not Found`. Instead, it returned a `200 OK` with the item's data, revealing the most critical bug of the system (**BUG-007**): the DELETE operation does not work.

### 4.3.3 Test Case Summary and CI/CD Selection

The CI/CD suite for these endpoints focuses on the happy paths and, most importantly, on regression testing for the critical deletion bug.

| Test Case Type | Description | Rationale for CI/CD Inclusion |
|---|---|---|
| GET Happy Path | Retrieve a valid, existing favorite item. | Ensures core data retrieval functionality works. |
| DELETE "Happy Path" | Delete a valid, existing item. | Confirms the API endpoint is reachable and returns the expected status code. |
| **Critical Bug Regression** | Attempt to retrieve an item immediately after deleting it. | **Most Critical Test**: This directly checks if the state transition bug (**BUG-007**) has been fixed. Its failure signals a major system issue. |

### 4.3.4 Execution Results and Illustration

This set of tests uncovered the most severe bugs in the application, primarily related to the non-functional DELETE operation and authorization flaws.

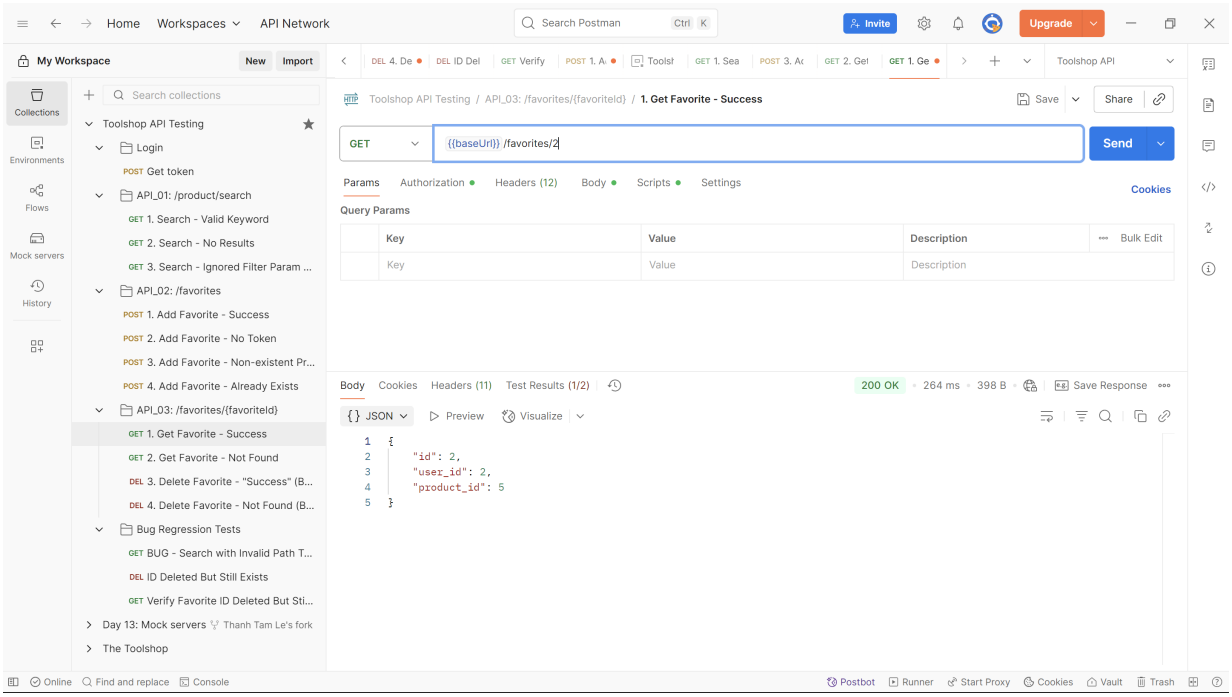



Figure 5: Successful execution of GET request for an existing favorite item.

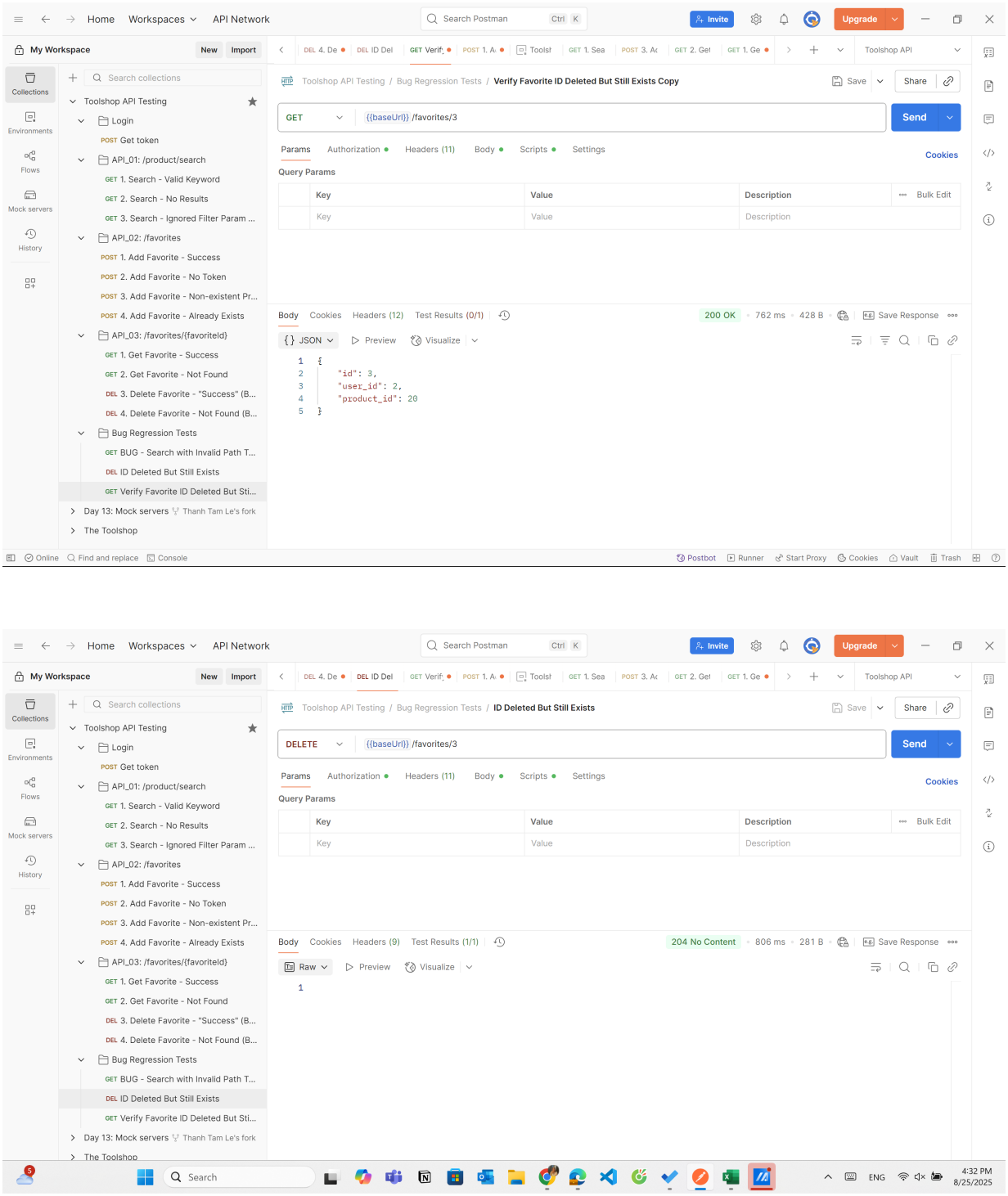






Figure 6: Critical Bug (BUG-007): A GET request for a "deleted" item returns a 200 OK, proving the DELETE operation failed.

# 5 Bug Reporting

Effective bug reporting is a cornerstone of the quality assurance process. It provides a formal method to document, track, and communicate defects discovered during testing. For this project, all identified issues were meticulously logged and detailed in a dedicated bug report file: `22127374_BugReport.xlsx`.

This section summarizes the findings from that report and provides illustrative examples of the logged defects.
A comprehensive log mapping every failed test case to its corresponding defect ID can also be found by cross-referencing the test suite in `22127374_TestCases.xlsx` with the aforementioned bug report file.

## 5.1 Summary of Discovered Bugs

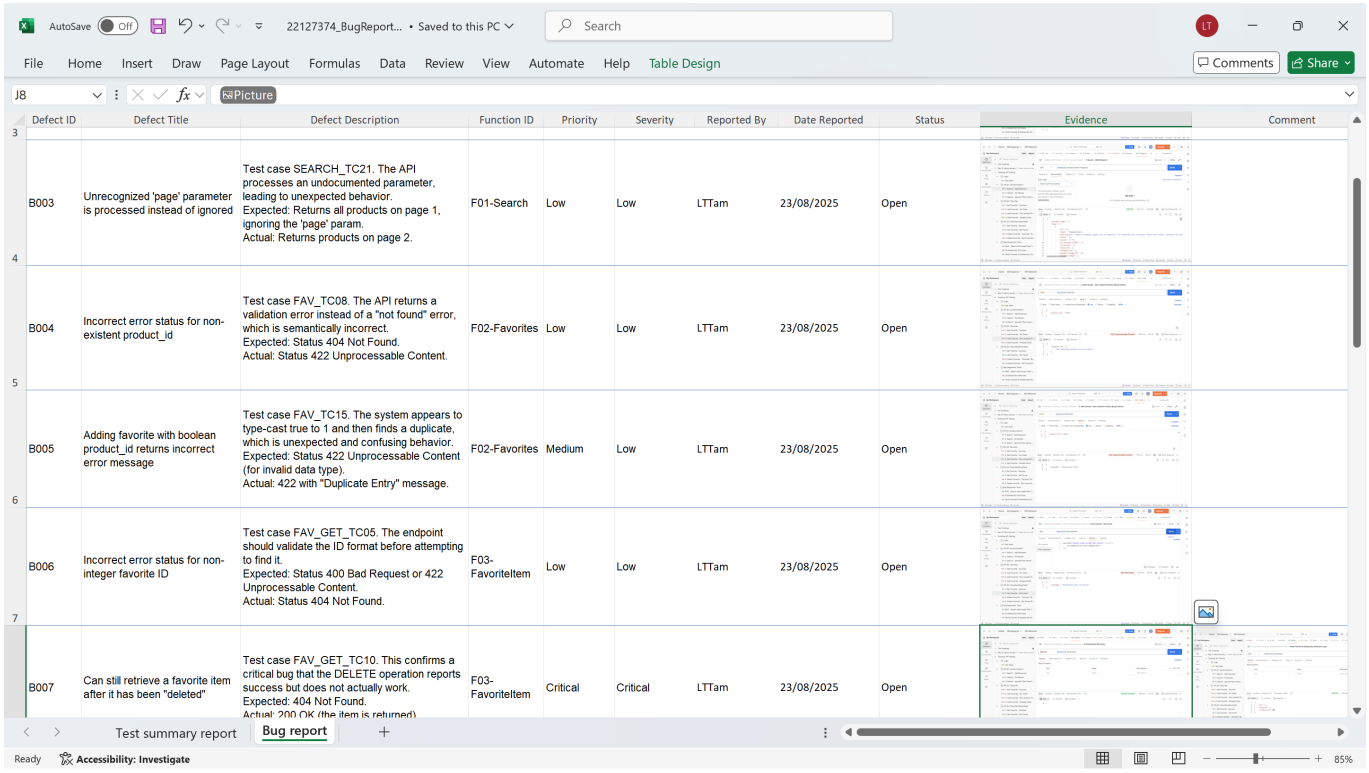
A total of **15 unique bugs** were identified and documented in the bug report file. These issues ranged in severity from minor semantic inconsistencies to critical security vulnerabilities and functional failures. The most significant finding was a critical flaw in the favorite item deletion logic, which rendered the entire feature non-functional and had a cascading impact on other tests.
The discovered bugs are categorized by severity in Table 5.1.

| Severity | Bug ID(s) | Brief Description of Issues |
|---|---|---|
| **Critical** | BUG-007, BUG-013 | Core functionality is completely broken (DELETE does not work) and a major security flaw was found (Authorization Bypass). |
| **High** | BUG-001, BUG-002, BUG-012, BUG-014 | Tests are blocked by critical bugs, major security vulnerabilities (Path Traversal, SQLi handling), and unhandled server errors (500). |
| **Medium** | BUG-005, BUG-008, BUG-009, BUG-010, BUG-011 | Incorrect business logic and misleading success responses for invalid operations. |
| **Low** | BUG-003, BUG-004, BUG-006, BUG-015 | Minor deviations from API specifications and incorrect, but not harmful, error responses. |

The most impactful issue, **BUG-007**, revealed that the `DELETE /favorites/{favoriteId}` endpoint, despite returning a `204 No Content` success status, failed to actually remove the item from the database. This critical failure blocked the execution of state transition tests like `FAV_TC07` (logged as **BUG-001**).

## 5.2 Illustrative Bug Reports

This subsection presents examples of the detailed bug reports as documented in the `22127374_BugReport.xls` file. Each entry in the report was structured to provide developers with all necessary information to understand, reproduce, and resolve the issue. This includes a unique Defect ID, a descriptive title, a clear description of the issue, and an assigned severity level.
Below are illustrative examples of the most critical bugs logged in the report.



| Defect ID | Defect Title | Defect Description | Function ID | Priority | Severity | Reported By | Date Reported | Status | Evidence | Comment |
|---|---|---|---|---|---|---|---|---|---|---|
| B003 | Unrecognized 'page' parameter is processed instead of ignored | Test case SEARCH_TC21. The API processes an undocumented parameter, leading to unexpected behavior.<br>Expected: Returns all products on page 1, ignoring the 'page' param.<br>Actual: Returns products on page 2. | F01-Search | Low | Low | LTTam | 23/08/2025 | Open | | |
| B004 | Incorrect error status for non-existent product_id | Test case FAV_TC08. The API returns a validation error instead of a "not found" error, which is semantically incorrect.<br>Expected: Status 404 Not Found.<br>Actual: Status 422 Unprocessable Content. | F02-Favorites | Low | Low | LTTam | 23/08/2025 | Open | | |
| B005 | Adding favorite with boolean product_id returns incorrect error message | Test case FAV_TC15. The system appears to type-cast true to 1 and then finds a duplicate, which is unexpected and confusing behavior.<br>Expected: Status 422 Unprocessable Content (for invalid data type).<br>Actual: 422 with "Duplicate Entry" message. | F02-Favorites | Medium | Low | LTTam | 23/08/2025 | Open | | |
| B006 | Incorrect error status for non-integer favorite ID | Test case FAV_GET_TC05. The endpoint should validate the ID format before attempting to find it.<br>Expected: Status 400 Bad Request or 422 Unprocessable Content.<br>Actual: Status 404 Not Found. | F03-FavoriteItem | Low | Low | LTTam | 23/08/2025 | Open | | |
| B007 | Can still retrieve a favorite item after it has been "deleted" | Test case FAV_GET_TC10. This confirms a critical bug that the DELETE operation reports success but does not actually work.<br>Expected: 404 Not Found.<br>Actual: 200 OK (the item is returned) | F03-FavoriteItem | Critical | Critical | LTTam | 23/08/2025 | Open | | |

Figure 7: Excel Report Entry - Critical Functional Failure.

The complete and detailed list of all 15 bug reports can be found in the accompanying `22127374_BugReport.xl` file.)

# 6 CI/CD Integration

To ensure long-term quality and prevent regressions, the API test suite was integrated into a Continuous Integration (CI) pipeline. The primary goal of this integration is to automatically execute a core set of tests every time new code is committed, providing rapid feedback to developers and maintaining a high standard of software reliability.

## 6.1 Strategy and Tools

The strategy for CI integration focused on balancing comprehensive coverage with execution speed. Instead of running the full suite of over 90 test cases, a smaller, targeted **smoke and regression test suite** was curated. This suite includes test cases covering:

- The primary "happy path" for each API.
- Critical negative paths, especially for security and authorization.
- Regression tests for the most severe bugs discovered.

This approach ensures that the CI pipeline can complete in a few minutes, providing fast and actionable feedback.

The following tools were used to build the pipeline:

- **GitHub Actions:** Chosen as the CI orchestrator due to its seamless integration with the source code repository.
- **Newman:** The command-line companion to Postman, used to run the test collection in a headless environment.

## 6.2 Workflow Configuration in GitHub Actions

A GitHub Actions workflow was defined in the file `.github/workflows/api-tests.yml`. This workflow is configured to trigger automatically on every push to the `main` branch and can also be run manually via `workflow_dispatch`.

The workflow consists of a single job that performs the following steps:

1. **Checkout code:** Downloads the latest version of the repository code, which includes the Postman collection and environment files.
2. **Set up Node.js:** Prepares the necessary runtime environment for Newman.
3. **Install Newman:** Installs the Newman CLI tool onto the runner.
4. **Run Postman tests:** Executes the API tests using the collection and environment files stored locally in the repository.

The complete workflow configuration is shown below:

```
name: Toolshop API Tests

on:
  push:
    branches: [ main ]
  workflow_dispatch:

jobs:
  run-api-tests:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v3

      - name: Set up Node.js
        uses: actions/setup-node@v3
        with:
          node-version: '18'

      - name: Install Newman
        run: npm install -g newman

      - name: Run Postman tests
        run: newman run postman/toolshop-api-tests.json -e postman/toolshop.postman_enviro
```

The complete workflow file and the project repository can be viewed live at the following link: https://github.com/thenhtemle/practice-software-testing

## 6.3 Automated Execution Results

The live results and all workflow runs can be accessed directly on the project's GitHub Actions tab: https://github.com/thenhtemle/practice-software-testing/actions/workflows/api-tests.yml

Once configured, the workflow runs automatically on every push to the `main` branch, and the results are logged directly within the GitHub Actions interface. The primary purpose of this pipeline is to act as a safety net by automatically detecting regressions or existing bugs.

The detailed output from Newman, as shown in Figure 8, provides a clear summary of the test run, including a list of any failed assertions. In this execution, the log clearly indicates that several tests failed, successfully catching known bugs before they could be overlooked. This demonstrates the effectiveness of the CI pipeline in automatically validating the API's health and preventing faulty code from progressing.

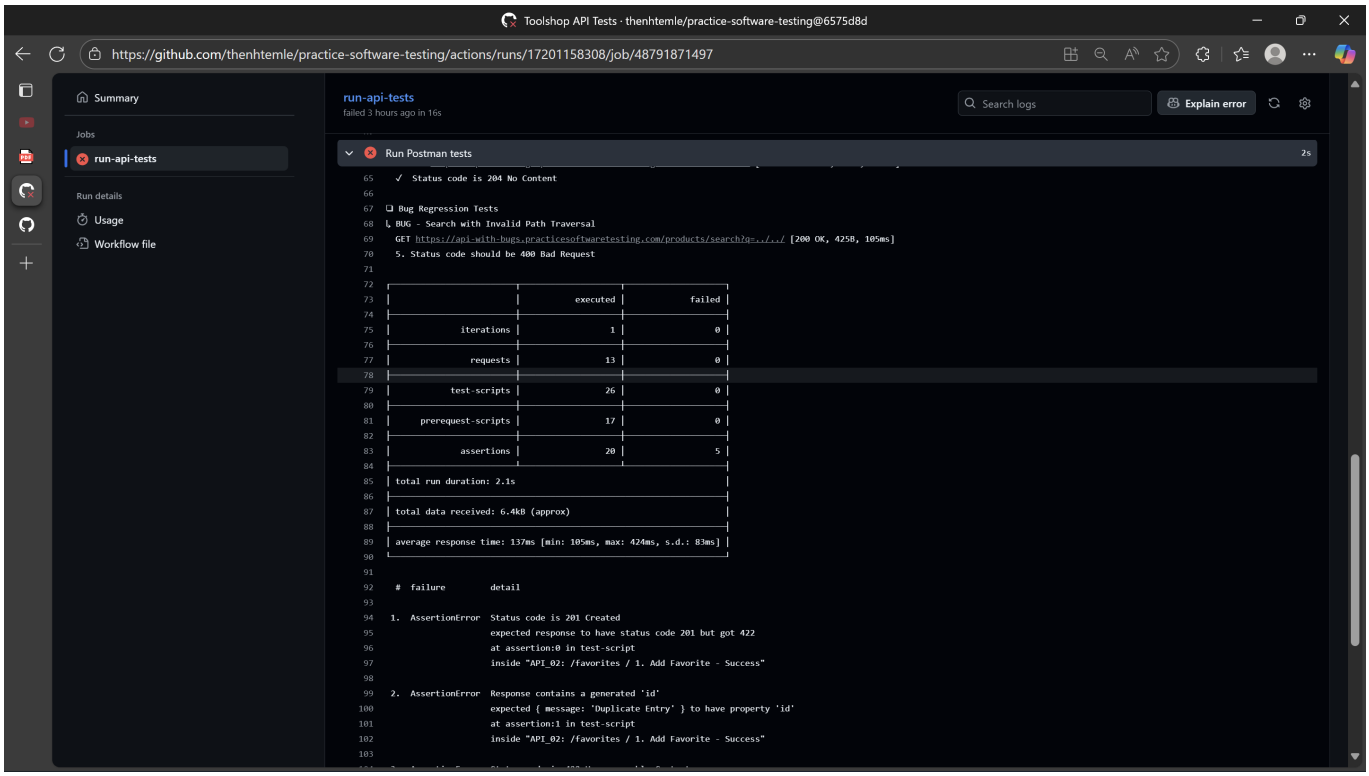



Figure 8: Detailed Newman test summary in the GitHub Actions log.

# 7 AI/LLM Tool Usage

In line with modern software development and testing practices, AI/LLM tools were leveraged throughout this project to enhance productivity, improve the quality of test design, and assist in documentation. These tools were used not as a replacement for critical thinking but as an intelligent assistant.

## 7.1 Description of AI/LLM Application

The application of AI/LLM tools can be categorized into four main areas:

- **Test Case Ideation:** The primary use was to brainstorm a comprehensive set of test scenarios. By providing the AI with an API's specification, it was able to suggest numerous edge cases, negative tests, and security-related inputs (e.g., XSS, SQLi payloads) that might otherwise be overlooked.
- **Script and Code Generation:** AI was used to generate boilerplate code for Postman's JavaScript-based test assertions. This accelerated the process of writing tests to validate complex JSON responses, check headers, and measure response times.
- **Documentation and Refinement:** The AI served as a powerful tool for drafting and refining the text in this report, helping to ensure the language was clear, professional, and technically accurate. It also assisted in converting content between formats (e.g., Markdown to LaTeX).
- **Debugging and Explanation:** When encountering errors in Postman scripts or the GitHub Actions YAML file, the AI was used as a first-line debugging tool, often providing quick insights into syntax errors or logical flaws.

## 7.2 List of Useful Prompts

Below is a curated list of prompts that proved to be particularly effective during this assignment.

**For Test Case Ideation:**

```
I am testing a REST API endpoint: "POST /favorites".
It requires authentication and takes a JSON body: {"product_id": <integer>}.
The product_id must be a valid and existing ID from the 'products' table.

Generate a comprehensive list of negative and edge case test scenarios for this endpoint.
Categorize them into:
1.  Authentication/Authorization Tests
2.  Request Body & Syntax Tests
3.  Data Type & Value Tests (for product_id)
4.  Business Logic Tests
```

**For Postman Script Generation:**

```
Write a Postman test script in JavaScript to verify the following in a JSON response:
1. The HTTP status code is exactly 201.
2. The response body contains a JSON object.
3. The response JSON object has a key named "id" which is a number.
4. The response JSON object has a key named "product_id" which is equal to the
   "product_id" variable from my Postman collection.
```

**For Documentation and Explanation:**

```
Explain the concept of State Transition Testing in the context of API testing.
Provide a clear example using a "favorites" feature where a user can
add and remove a favorite item. Describe the states and transitions involved.
```

# 8 Conclusion

This final section summarizes the work accomplished throughout the API testing project for The Toolshop application. It also reflects on the key challenges encountered and the valuable lessons learned from the experience.

## 8.1 Summary of Work

This project successfully executed a comprehensive API testing process, fulfilling all the requirements outlined in the assignment. A test suite of over 90 granular test cases was systematically designed and executed across three high-priority API endpoints: product search, adding favorites, and managing favorite items.

The rigorous testing effort led to the discovery and documentation of **15 unique bugs**. These defects ranged in severity from minor inconsistencies to critical functional and security flaws, providing valuable feedback on the application's stability. Key findings were meticulously documented in the `22127374_BugReport.xlsx` file to facilitate resolution.

A significant achievement of this project was the successful implementation of an automated regression test suite. By integrating a curated Postman collection with a **GitHub Actions CI/CD pipeline**, a foundational safety net was established. This ensures that core API functionalities are automatically validated upon every code change, effectively preventing future regressions.

## 8.2 Challenges and Lessons Learned

Several challenges were encountered during the project, which in turn provided important learning opportunities.

**Key Challenges:**

- **Critical Bug as a Test Blocker:** The most significant challenge was the non-functional `DELETE /favorites/{favoriteId}` API (**BUG-007**). This critical defect acted as a major blocker, preventing the complete execution of the end-to-end state transition test plan (**FAV_TC07**) for the favorites feature.
- **API Specification Ambiguity:** In some cases, the API's behavior deviated from RESTful best practices. For instance, the system returned a `422 Unprocessable Content` status instead of a more appropriate `404 Not Found` for a non-existent resource (**BUG-004**). This highlighted the challenge of testing without a precise and up-to-date API specification.

**Lessons Learned:**

- **The Importance of End-to-End State Testing:** The critical delete bug underscored the necessity of testing the full lifecycle of a feature (Create, Read, Update, Delete). Verifying individual operations in isolation is not enough; their interactions must also be validated.
- **The Value of a CI/CD Safety Net:** The process of setting up the CI pipeline demonstrated its immense value. Even a small set of automated tests can provide significant confidence and catch regressions early, long before they could impact end-users.

- **Systematic Bug Reporting is Key:** Documenting bugs in a structured format, complete with clear reproduction steps, is crucial for effective communication with development teams and ensures that issues can be addressed efficiently.
- **AI as a Productivity Multiplier:** Leveraging AI/LLM tools for brainstorming test cases, generating scripts, and refining documentation proved to be a highly effective strategy. It significantly accelerated the workflow and improved the overall quality of the project deliverables.

# 9 Self-Assessment

| Criteria | Outcomes (Brief description about what you get/trouble from each requirement) | Percent | Self-Assessed Grade |
|---|---|---|---|
| **1** | **API 1** | **30%** | |
| | 1.1 Report | 15 | 15 |
| | 1.2 Test Cases (30) | 5 | 5 |
| | 1.3 Screenshots on Testing Tools | 5 | 5 |
| | 1.4 Bugs | 5 | 5 |
| **2** | **API 2** | **30%** | |
| | 2.1 Report | 15 | 15 |
| | 2.2 Test Cases (30) | 5 | 5 |
| | 2.3 Screenshots on Testing Tools | 5 | 5 |
| | 2.4 Bugs | 5 | 5 |
| **3** | **API 3** | **30%** | |
| | 3.1 Report | 15 | 15 |
| | 3.2 Test Cases (30) | 5 | 5 |
| | 3.3 Screenshots on Testing Tools | 5 | 5 |
| | 3.4 Bugs | 5 | 5 |
| **4** | **Well formatted** | **10%** | **10%** |
| | **Total** | **100%** | **100%** |